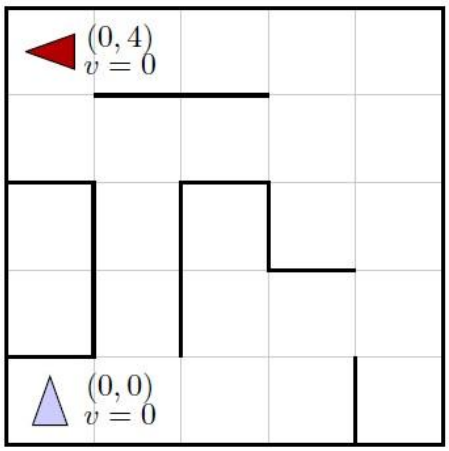
BÀI TẬP TÌM KIẾM VÀ HEURISTIC TRONG AI

Họ và tên	Võ Thị Như Ý
MSSV / Lớp	3123410439/DTN1241

Mục tiêu học tập

- Mô hình hóa bài toán tìm kiếm trong AI.
- Vận dụng BFS, UCS, Greedy và A*.
- Phân tích và đánh giá heuristic.
- Phân biệt heuristic admissible và consistent.
- So sánh tính đầy đủ, tối ưu và hiệu quả của các thuật toán tìm kiếm.

Câu 1. Tìm kiếm và heuristic



Một tác tử dạng xe cần thoát khỏi mê cung như hình trên. Tác tử có hướng và tại mọi thời điểm luôn quay về một trong bốn hướng d ∈ Bắc, Nam, Đông, Tây. Trong mỗi bước, tác tử có thể tiến về phía trước với vận tốc có thể điều chỉnh hoặc thực hiện thao tác rẽ.

Các hành động di chuyển gồm tăng tốc, giữ tốc độ và giảm tốc. Sau hành động, tác tử di chuyển một số ô bằng vận tốc mới v'. Gọi v là vận tốc hiện tại và v' là vận tốc sau điều chỉnh:

- Tăng tốc (Faster): v' = v + 1
- Giảm tốc (Slower): v' = v - 1
- Giữ tốc độ (Maintain): v' = v

Các hành động rẽ gồm trái và phải, làm thay đổi hướng 90°. Chỉ được phép rẽ khi vận tốc bằng 0; sau khi rẽ vận tốc vẫn bằng 0.

- Rẽ trái: quay 90° ngược chiều kim đồng hồ.
- Rẽ phải: quay 90° theo chiều kim đồng hồ.

Ví dụ: nếu tác tử ở (0,0), quay về Bắc, v = 0 và cần đến (2,0), quay về Đông, v = 0 thì một chuỗi hành động hợp lệ là: rẽ phải, tăng tốc, giữ tốc độ, giảm tốc.

Các hành động không hợp lệ gồm:

- Hành động làm tác tử va vào tường.
- Làm $v < 0$ hoặc $v > Vmax$.
- Giữ tốc độ khi $v = 0$.
- Rẽ khi $v \neq 0$.

Mục tiêu là tìm kế hoạch đưa xe dừng hẳn ($v = 0$) tại ô thoát, đúng hướng đích, với số hành động ít nhất. Chi phí đường đi được tính bằng số hành động đã thực hiện.

a) Giả sử tác tử đi theo nhánh ngoài cùng bên trái (đi qua ô (1,2)), từ trạng thái bắt đầu (0,0), hướng Bắc đến trạng thái đích (0,4), hướng Tây. Hãy viết chuỗi hành động ngắn nhất.

Chuỗi hành động (15 bước): Rẽ phải → Tăng tốc → Giảm tốc → Rẽ trái → Tăng tốc → Giữ tốc độ → Giữ tốc độ → Giảm tốc → Rẽ trái → Tăng tốc → Giảm tốc → Rẽ phải → Tăng tốc → Giảm tốc → Rẽ trái.

b) Nếu lưới có kích thước $M \times N$ và vận tốc tối đa là Vmax, kích thước không gian trạng thái là bao nhiêu? Giả sử mọi cấu hình đều có thể đạt được từ trạng thái bắt đầu.

Mỗi trạng thái $s = (x, y, d, v)$ gồm:

Tọa độ (x, y): Có $M \times N$ vị trí.

Hướng d € Bắc, Nam, Đông, Tây: Có 4 hướng.

Vận tốc v € 0, 1, V_{max}, Có V_{max+1} giá trị

Kích thước không gian trạng thái: $4 \times M \times N \times V_{max+1}$

c) Một "nút con" của trạng thái s là trạng thái s' có thể đạt được bằng một hành động hợp lệ từ s. Có thể tồn tại trạng thái không có nút con hay không? Nếu có, hãy cho ví dụ; nếu không, hãy giải thích.

☒ Có ☐ Không

Ví dụ: Nếu tác tử đang di chuyển với vận tốc $v > 0$ và ngay trước mặt là tường chắn. Vì $v > 0$ nên tác tử không thể chọn các hành động Rẽ (chỉ được rẽ khi v=0). Các hành động Tăng/Giữ/Giảm tốc đều khiến xe tiến về phía trước và va vào tường (hành động không hợp lệ). Do đó, trạng thái này không có nút con hợp lệ nào.

d) Hệ số phân nhánh lớn nhất của bài toán là bao nhiêu? Hãy cho một trạng thái ví dụ (x, y, hướng, vận tốc) đạt hệ số phân nhánh này và liệt kê các hành động có thể thực hiện.

Hệ số phân nhánh lớn nhất: 3

Ví dụ trạng thái: (x, y, Bắc, 0) khi ô phía trước thông thoáng.

Các hành động có thể thực hiện:

Khi v = 0: Rẽ trái, Rẽ phải, Tăng tốc (v'=1).

Khi v > 0: Tăng tốc (v'=v+1), Giữ tốc độ (v'=v), Giảm tốc (v'=v-1).

e) Khoảng cách Manhattan từ vị trí hiện tại của tác tử đến vị trí cửa ra có phải là heuristic chấp nhận được không? Nếu không, hãy cho một trạng thái phản ví dụ, nêu giá trị heuristic và chi phí thực tế tối ưu đến đích.

☐ Có ☒ Không

Phản ví dụ:

Heuristic hM(s) = $|x - x_{goal}| + |y - y_{goal}|$ tính theo số ô.

Giả sử $V_{max} = 3$. Xe ở vị trí (0,0) hướng Đông, v=2, đích ở (3,0). Xe chỉ cần 1 hành động Tăng tốc (v'=3) là tiến thẳng 3 ô tới (3,0), do đó chi phí thực tế h^*(s) = 1.

Giá trị heuristic hM(s) = |0-3| + |0-0| = 3. Vì hM(s) = 3 > h^*(s) = 1 (đánh giá cao hơn chi phí thực tế) nên hM không admissible.

f) Heuristic h = khoảng cách Manhattan / Vmax có phải là admissible không? Giải thích. Nếu không, hãy cho phản ví dụ tương tự câu (e).

☒ Có ☐ Không

Giải thích:

-Trong mỗi bước di chuyển, tác tử tiến xa nhất được V_{max} ô.

-Để đi hết khoảng cách Manhattan $D = |x - x_{goal}| + |y - y_{goal}|$, số bước di chuyển tối thiểu tuyệt đối phải là $\lceil D / V_{max} \rceil \geq D / V_{max}$.

-Các thao tác rẽ, hãm phanh về 0 hay vượt qua tường ngăn chỉ làm tăng thêm số bước thực tế h^*(s).

-Vì $h(s) = D / V_{max} \leq h^*(s)$ với mọi trạng thái s, nên h là admissible.

g) Nếu dùng heuristic không admissible trong A* Tree Search, tính đầy đủ của thuật toán có thể thay đổi không? Giả sử đồ thị hữu hạn và heuristic không âm.

☐ Có ☒ Không

Giải thích: Đồ thị hữu hạn và $h(s) \geq 0$, chi phí mỗi bước dương nên A* vẫn mở rộng các nút theo thứ tự f(n) tăng dần cho đến khi duyệt hết đồ thị hoặc tìm thấy đích, tính đầy đủ không bị ảnh hưởng.

h) Nếu dùng heuristic không admissible trong A* Tree Search, tính tối ưu của thuật toán có thể thay đổi không? Giả sử đồ thị hữu hạn và heuristic không âm.

☒ Có ☐ Không

Giải thích: Khi heuristic không admissible, h(n) có thể phóng đại chi phí làm cho đường đi tối ưu bị trì hoãn mở rộng, A* có thể trả về một đường đi dưới tối ưu (suboptimal).

i) Những lý do nào sau đây có thể hợp lý khi dùng heuristic không admissible thay vì admissible? Chọn tất cả phương án phù hợp.

☒ Heuristic không admissible có thể dễ tính hơn, giúp giảm thời gian tính heuristic.

☒ Heuristic không admissible có thể gần chi phí thực hơn (dù đôi khi đánh giá cao), nhờ đó có thể mở ít nút hơn.

☐ Heuristic không admissible vẫn luôn tìm được đường đi tối ưu khi mọi chi phí thực không âm.

☒ Heuristic không admissible có thể được dùng để làm thuật toán hoàn toàn không xét một phần của đồ thị.

Câu 2. Các nút tìm kiếm

Xét tree search (không dùng tập explored) cho một bài toán tìm kiếm bất kỳ có hệ số phân nhánh tối đa b. Mỗi nút tìm kiếm n có chi phí tích lũy g(n), heuristic admissible h(n) và độ sâu d(n). Gọi nc là nút đích có chi phí nhỏ nhất và ns là nút đích nông nhất.

Với mỗi trường hợp, hãy viết một bất đẳng thức mô tả tập các nút được mở rộng trước khi thuật toán kết thúc. Không cần xử lý trường hợp hòa (tie). Nếu không có nút nào thỏa, ghi "không có". **a)**

Breadth-First Search (BFS).

Bất đẳng thức: $d(n) < d(n_s)$

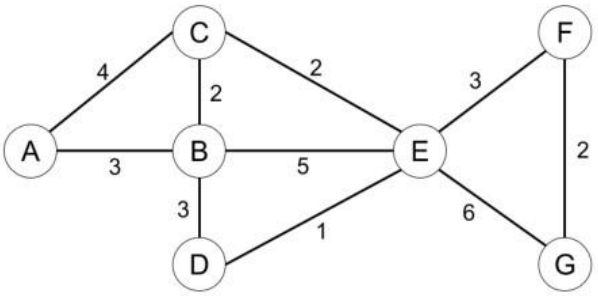
b) Uniform-Cost Search (UCS).

Bất đẳng thức: $g(n) < g(n_c)$

c) A* Search, giả sử h là heuristic nhất quán.

Bất đẳng thức: $g(n) + h(n) < g(n_c)$

Câu 3. Tìm kiếm trên đồ thị



Node	h_1	h_2
A	12	11
B	6	7
C	9	6
D	3	4
E	3	5
F	2	1
G	0	0

Nút	h1	h2
A	12	11
B	6	7
C	9	6
D	3	4
E	3	5
F	2	1
G	0	0

Xét đồ thị vô hướng như hình. A là trạng thái bắt đầu và G là trạng thái đích. Chi phí của mỗi cạnh được ghi trên đồ thị. Bảng trên cung cấp hai heuristic h1 và h2. Khi có nhiều nút cùng ưu tiên, kết quả có thể phụ thuộc quy tắc phá hòa (tie-breaking).

a) Với mỗi chiến lược tìm kiếm đồ thị dưới đây, hãy đánh dấu X vào mọi đường đi mà thuật toán có thể trả về. Nếu kết quả không nằm trong các đường đi liệt kê, ghi đường đi đúng ở cột "Khác".

Thuật toán	A-C-E-G	A-C-E-F-G	A-B-D-E-F-G	Khác
UCS	☒	☒	☐	☐
Greedy với h1	☐	☐	☐	A-B-D-G
Greedy với h2	☒	☐	☐	☐
A* với h1	☐	☒	☐	☐
A* với h2	☒	☒	☐	☐

b) Chi phí của đường đi tối ưu từ A đến G theo UCS là bao nhiêu?

Đáp án: 7

c) Heuristic h1 có admissible không? Có consistent không? Admissible: ☐ Có ☒ Không Consistent: ☐ Có ☒ Không

d) Heuristic h2 có admissible không? Có consistent không?

Admissible: ☐ Có ☒ Không Consistent: ☐ Có ☒ Không

Câu 4. Trắc nghiệm và trả lời ngắn về tìm kiếm

a) Đánh dấu Đúng/Sai. Giả sử mọi hành động có chi phí ít nhất ε, với ε > 0, và mọi heuristic được dùng đều consistent.

Depth-First Tree Search trên một đồ thị hữu hạn luôn được đảm bảo là complete.

☐ Đúng ☒ Sai

Breadth-First Tree Search trên một đồ thị hữu hạn luôn được đảm bảo là complete.

☒ Đúng ☐ Sai

Iterative Deepening Tree Search trên một đồ thị hữu hạn luôn được đảm bảo là complete.

☒ Đúng ☐ Sai

Với mọi đồ thị không có chu trình, Graph Search luôn có frontier lớn hơn Tree Search.

☐ Đúng ☒ Sai

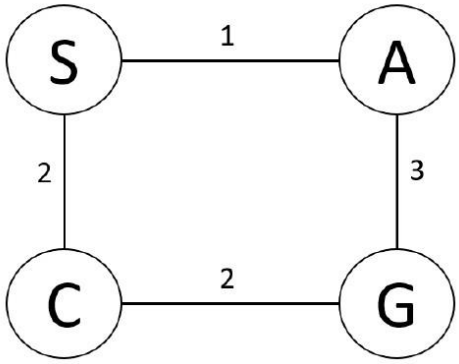
Iterative Deepening Graph Search có độ phức tạp thời gian của BFS và độ phức tạp bộ nhớ của DFS.

☒ Đúng ☐ Sai

Nếu h1(s) và h2(s) đều là heuristic consistent thì min(h1(s), h2(s)) cũng nhất thiết consistent.

☒ Đúng ☐ Sai

b) Xét đồ thị trạng thái dưới đây, S là bắt đầu và G là đích. Hãy điền các giá trị heuristic sao cho heuristic admissible nhưng không consistent.



Heuristic Function	
State	$h(s)$
S	
A	
C	
G	0

Trạng thái	h(s)
S	3
A	0
C	1
G	0

--- Hết ---

BÀI TẬP TÌM KIẾM TRONG AI

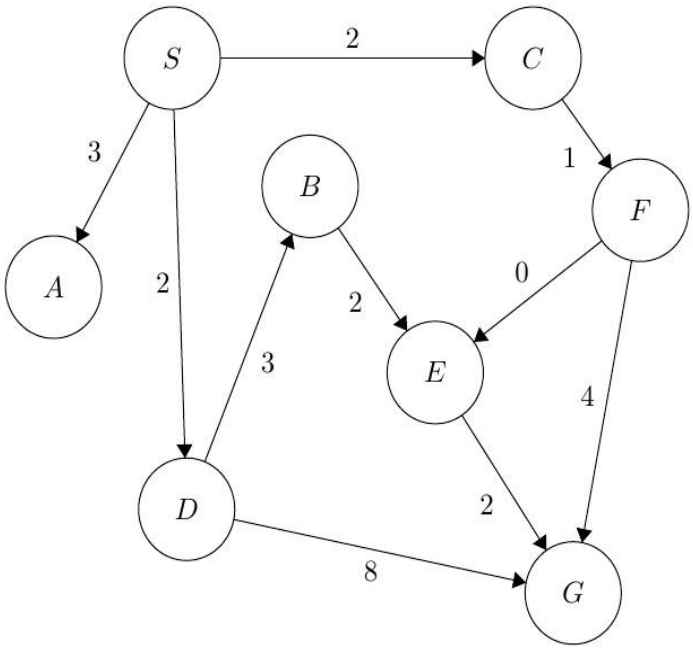
Họ và tên	Võ Thị Như Ý
MSSV / Lớp	3123410439/ DTN1241

Mục tiêu học tập

- Mô phỏng các thuật toán BFS, DFS và Iterative Deepening.
- Biểu diễn bài toán Tháp Hà Nội dưới dạng bài toán tìm kiếm.
- Thiết kế và phân tích heuristic cho A*.
- So sánh DFS, BFS, UCS, Greedy Search và A* trên cùng một bài toán.
- Phân tích các phát biểu lý thuyết về heuristic, A*, Minimax và Expectimax.

Bài 1. Các thuật toán tìm kiếm

Cho đồ thị có trạng thái bắt đầu S và trạng thái đích G như hình dưới đây.



Hình 1. Đồ thị dùng cho các thuật toán tìm kiếm

Yêu cầu: Với mỗi thuật toán, hãy ghi thứ tự các nút được đưa vào explored set, đường đi được trả về và tổng chi phí của đường đi. Nếu có hòa, phá hòa theo thứ tự alphabet của trạng thái cuối cùng trong đường đi.

1.1. Breadth-First Search (BFS)

Nội dung	Kết quả
Explored Set	S,A,C,D,F,B,G
Path	$S \rightarrow D \rightarrow G$
Total Cost	$2 + 8 = 10$

1.2. Depth-First Search (DFS)

Nội dung	Kết quả
Explored Set	S,A,C,F,E,G
Path	$S \to C \to F \to E \to G$
Total Cost	$2 + 1 + 0 + 2 = 5$

1.3. Iterative Deepening Search (IDS)

Giới hạn độ sâu	Các nút được duyệt
0	S
1	S,A,C,D
2	S,A,C,F,D,B,G

Path cuối cùng: S→D→G

Total Cost: 2+8=10

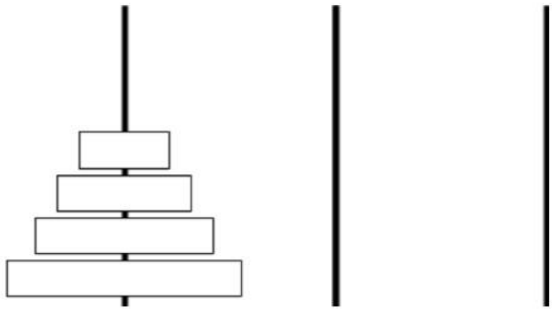
Câu hỏi thảo luận: BFS và DFS khác nhau ở cấu trúc frontier và tính tối ưu như thế nào?

Trả lời:

- Cấu trúc Frontier: BFS dùng Queue (FIFO) giúp quét theo từng tầng ngang, trong khi DFS dùng Stack (LIFO) giúp đi sâu vào từng nhánh.
- Tính tối ưu: BFS luôn tìm được con đường ngắn nhất về số bước (độ sâu) trong đồ thị chi phí bằng nhau (unweighted graph). DFS không đảm bảo tính tối ưu (có thể trả về con đường vòng rất xa).

Bài 2. Tháp Hà Nội - Tower of Hanoi

Bài toán gồm n đĩa có kích thước khác nhau và ba cọc. Ban đầu các đĩa được xếp trên một cọc theo thứ tự từ lớn đến nhỏ. Mỗi lần chỉ được di chuyển một đĩa và không được đặt đĩa lớn lên trên đĩa nhỏ. Mục tiêu là chuyển toàn bộ đĩa sang cọc thứ ba.



Hình 2. Trạng thái ban đầu của bài toán Tháp Hà Nội

2.1. Biểu diễn trạng thái

Hãy đề xuất cách biểu diễn một trạng thái của bài toán. Biểu diễn phải cho biết mỗi đĩa đang ở cọc nào và đủ để xác định các hành động hợp lệ.

Đánh số các đĩa từ 1 đến n theo thứ tự kích thước tăng dần (đĩa 1 nhỏ nhất, đĩa n lớn nhất). Đặt tên 3 cọc là A,B,C. Một trạng thái s được biểu diễn bằng một bộ (tuple) có n phần tử:

$$s=(p_1,p_2,...,p_n)$$

trong đó $p_i \in \{A,B,C\}$ biểu thị cọc hiện tại của đĩa thứ i.

Do các đĩa nằm trên cùng một cọc bắt buộc phải tuân theo thứ tự đĩa nhỏ xếp trên đĩa lớn, nên vị trí chính xác của từng đĩa trên cọc được xác định duy nhất dựa trên danh sách s.

2.2. Kích thước không gian trạng thái

Với n đĩa, hãy xác định số trạng thái có thể có theo cách biểu diễn của bạn và giải thích cách suy luận.

Số trạng thái có thể có: 3^n

Mỗi đĩa i (i=1,…,n) độc lập có 3 lựa chọn đặt vào cọc (A, B, hoặc C). Như vậy, tổng số cấu hình trạng thái phân biệt là (3×3×⋯×3)=3n.

2.3. Trạng thái bắt đầu

Hãy viết trạng thái khởi đầu S0 theo cách biểu diễn đã chọn.

Ban đầu cả n đĩa đều xếp trên cọc A:

$$S_0=\underbrace{A, A, \dots, A}_{\text{n lần}}$$

2.4. Các hành động hợp lệ

Từ một trạng thái bất kỳ, hãy mô tả tập Actions(s). Một hành động hợp lệ phải thỏa mãn các điều kiện nào?

Mô tả tập Actions(s): Hành động có dạng (I, cọc nguồn, cọc đích), đại diện cho việc chuyển đĩa i từ cọc nguồn sang cọc đích.

Điều kiện hợp lệ:

1. Đĩa i phải là đĩa nằm trên cùng (đĩa nhỏ nhất) của cọc nguồn.
2. Cọc đích đang trống hoặc đĩa trên cùng hiện tại của cọc đích phải lớn hơn đĩa i (đảm bảo không đặt đĩa lớn hơn lên trên đĩa nhỏ hơn).

2.5. Goal Test

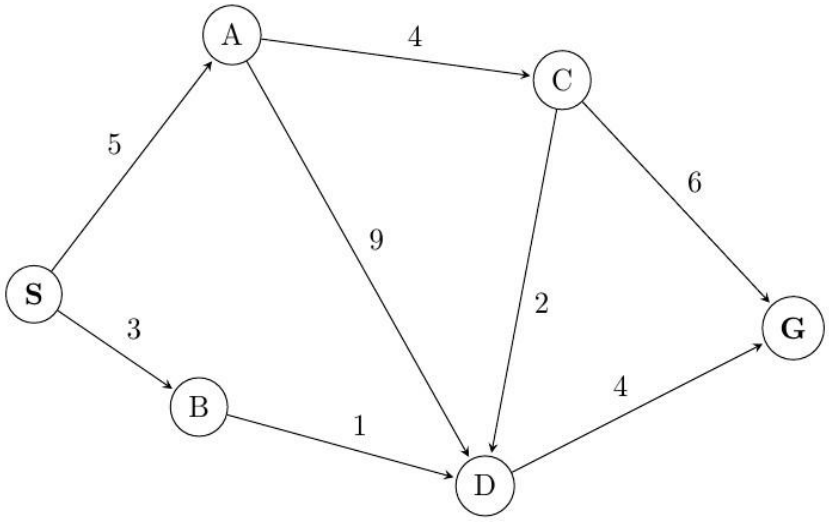
Hãy mô tả hàm goalTest(state): khi nào hàm trả về True?

Mục tiêu là chuyển toàn bộ n đĩa sang cọc thứ ba (C).

$$\text{goalTest(s)} \begin{cases} True & \text{nếu } s = \underbrace{(C, C, C, \dots, C)}_{n \text{ lần}} \\ False & \text{ngược lại} \end{cases}$$

Bài 3. Thiết kế và phân tích Heuristic

Xét đồ thị dưới đây. Hãy thiết kế heuristic và phân tích ảnh hưởng của heuristic đến thuật toán A*.



Hình 3. Đồ thị dùng để thiết kế heuristic

Công thức A*: f(n) = g(n) + h(n), trong đó g(n) là chi phí thực tế từ S đến n và h(n) là chi phí ước lượng từ n đến đích.

3.1. Thiết kế heuristic admissible và consistent

Điền giá trị h(n) cho từng nút sao cho heuristic phù hợp với yêu cầu của nhóm.

Node	h(n)
S	8
A	10
B	5
C	6
D	4
G	0

3.2. Chạy A* với heuristic đã thiết kế

Explored Set: [S,B,D,G]

Path: S→B→D→G

Total Cost: 8

3.3. Heuristic admissible nhưng không consistent

Đề xuất một bộ giá trị h(n) admissible nhưng không consistent. Chỉ ra ít nhất một cạnh vi phạm điều kiện h(n) <= c(n,n') + h(n').

Bộ giá trị đề xuất:

h(S)=8

h(A)=10

$h(B)=0$

$h(C)=6$

$h(D)=4$

$h(G)=0$

Cạnh vi phạm điều kiện Consistency: Xét cạnh (S,B) có chi phí $c(S,B)=3$:

$h(S)\leq c(S,B)+h(B)\Leftrightarrow 8\leq 3+0=3$(Sai)

Do đó, cạnh (S,B) vi phạm điều kiện $h(n)\leq c(n,n')+h(n')$.

3.4. Câu hỏi mở rộng

Giải thích vì sao một heuristic consistent thì cũng admissible, giả sử h(G) = 0.

Định nghĩa Consistency: Với mọi cạnh $n\rightarrow n'$ có chi phí $c(n,n')$:

$h(n)\leq c(n,n')+h(n')$

Cộng tích lũy dọc theo đường đi ngắn nhất: Giả sử đường đi tối ưu từ n đến G là $n=n_0\rightarrow n_1\rightarrow\cdots\rightarrow n_k=G$:

$h(n_0)\leq c(n_0,n_1)+h(n_1)$

$h(n_1)\leq c(n_1,n_2)+h(n_2)$

$\vdots$

$h(n_{k-1})\leq c(n_{k-1},G)+h(G)$

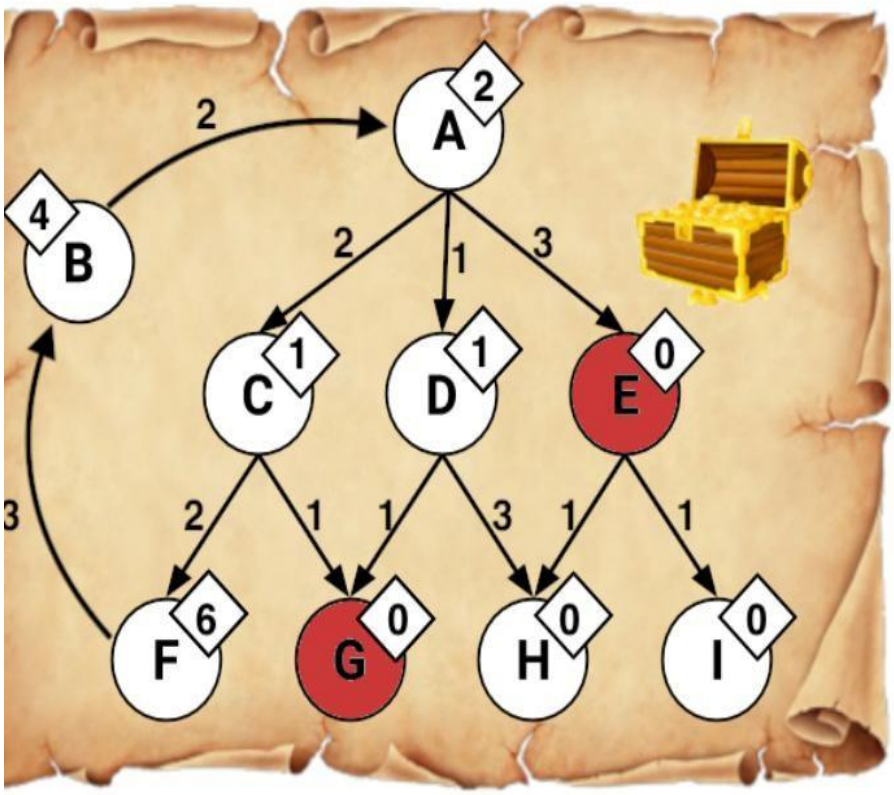
Cộng vế theo vế: Triệt tiêu các $h(n_1),h(n_2),\ldots,h(n_{k-1})$ ở hai vế, ta được:

$h(n)\leq\sum c(n_i,n_{i+1})+h(G)$

Kết luận: $h(n)\leq h*(n)$ với mọi $n\Longrightarrow$h thỏa mãn tính Admissible.

Bài 4. Treasure Hunting (Tìm kho báu)

A là đảo xuất phát. Các nút màu đỏ là các đảo có kho báu; chỉ cần tới một trong các đảo đó. Mũi tên biểu diễn hành động có hướng; số trên cạnh là action cost. Giá trị trong hình thoi là heuristic ước lượng chi phí tối thiểu từ nút hiện tại tới một kho báu.



Hình 4. Bản đồ tìm kho báu (giữ nguyên hình gốc)

Quy tắc phá hòa: khi lấy phần tử khỏi frontier, nếu hòa thì chọn theo thứ tự alphabet.

4.1. Depth-First Search

Explored Set: A,C,F,B,G

Path returned: A→C→G

4.2. Breadth-First Search

Explored Set: A,C,D,E

Path returned: A→E

4.3. Uniform-Cost Search

Explored Set: A,C,D,G

Path returned: A→D→G

Cost: 2

4.4. Greedy Search

Explored Set: A,E

Path returned: A→E

Cost: 3

4.5. A* Search

Explored Set: A,D,G

Path returned: A→D→G

Cost: 2

Bảng tổng hợp so sánh

Thuật toán	Tiêu chí chọn node	Path	Cost
DFS	Độ sâu	A→C→G	3
BFS	Độ sâu nhỏ nhất	A→E	3
UCS	g(n)	A→D→G	2
Greedy	h(n)	A→E	3
A*	g(n)+h(n)	A→D→G	2

Bài 5. Câu hỏi Đúng/Sai

Với mỗi phát biểu, hãy ghi Đúng hoặc Sai và giải thích ngắn gọn. Nếu sai, nên đưa ra phản ví dụ.

Câu 1. Depth-first search luôn mở rộng ít nhất nhiều node bằng A* sử dụng một admissible heuristic.

Đúng / Sai: Sai

Giải thích: DFS có thể may mắn đi đúng nhánh chứa đích ngay lập tức và dừng lại với rất ít nút được mở rộng, trong khi A∗ phải mở rộng thêm các nút thuộc các nhánh khác để đảm bảo tính tối ưu của đường đi.

Câu 2. Giả sử một quân xe trong cờ vua có thể đi bất kỳ số ô theo hàng hoặc cột trong một nước đi và không thể nhảy qua quân khác. Manhattan distance là một admissible heuristic cho số nước đi nhỏ nhất từ ô A đến ô B.

Đúng / Sai: Sai

Giải thích: Quân xe trên bàn cờ trống có thể di chuyển từ ô (x_1,y_1) đến ô $(x2_2,y_2)$ tối đa trong 2 nước đi (nếu khác cả hàng và cột) hoặc 1 nước đi (nếu cùng hàng/cột). Trong khi đó, khoảng cách Manhattan $|x_1-x_2|+|y_1-y_2|$ có thể lên tới 14 (trên bàn cờ 8×8). Vì $h(n)>h*(n)$ trong hầu hết các trường hợp, Manhattan distance không admissible cho di chuyển của quân xe.

Câu 3. Euclidean distance là một admissible heuristic cho bài toán tìm đường của Pacman.

Đúng / Sai: Đúng

Giải thích: Khoảng cách Euclidean (đường chim bay) luôn nhỏ hơn hoặc bằng khoảng cách di chuyển thực tế dọc theo lưới ô vuông có vật cản/tường trong bản đồ Pacman ($h_{Euclidean}\leq h*$). Do đó nó luôn là một admissible heuristic.

Câu 4. Tổng của nhiều admissible heuristics vẫn luôn là một admissible heuristic.

Đúng / Sai: Sai

Giải thích: Giả sử $h_1(n)$ và $h_2(n)$ đều admissible (tức $h_1\leq h*$ và $h_2\leq h*$). Tổng $h(n)=h_1(n)+h_2(n)$ có thể vượt quá $h*(n)$.

Phản ví dụ: Cho $h*(n)=5$, chọn $h_1(n)=3\leq 5$ và $h_2(n)=4\leq 5$. Khi đó $h_1+h_2=7>5=h*(n)$, vi phạm tính admissible.

Câu 5. Admissibility của một heuristic cho A* kéo theo consistency.

Đúng / Sai: Sai

Giải thích: Tính consistency kéo theo tính admissibility, nhưng chiều ngược lại **không luôn đúng**. Có nhiều heuristic admissible nhưng vi phạm điều kiện tam giác (consistency) trên từng cạnh

Câu 6. A* với graph search luôn tối ưu nếu heuristic admissible.

Đúng / Sai: Sai

Giải thích: A∗ với Graph Search (có sử dụng tập explored để không duyệt lại các đỉnh đã mở rộng) chỉ đảm bảo trả về lời giải tối ưu khi heuristic đạt tính consistency. Nếu heuristic chỉ dừng lại ở mức admissible, A∗ Graph Search có thể bỏ qua đường đi tối ưu hơn tới một đỉnh đã bị duyệt trước đó. (Để đảm bảo tính tối ưu chỉ với admissible heuristic, thuật toán phải dùng Tree Search hoặc cho phép reopening các nút trong Graph Search).

Tổng kết nhanh

Nhóm	Thuật toán	Hàm/tiêu chí chính
Uninformed Search	BFS / DFS / IDS	Không sử dụng heuristic
Cost-based Search	UCS	g(n)
Informed Search	Greedy	h(n)
Informed Search	A*	g(n)+h(n)

Ghi nhớ: A* dùng f(n) = g(n) + h(n).

---o0o---

(Hết)